

pa7uiclf

Ciência da Computação

Análise de Algoritmos com Listas (Simples e Dupla) Encadeadas

22/08/2025

Estrutura de dados - Listas

- **Revisão Paradigmas de Programação**
- **Listas simplesmente encadeadas**
- **Listas duplamente encadeadas**
- **Exercícios**
- **Caso de uso**

Estrutura de dados - Listas

- **Listas simplesmente encadeadas**

Estrutura de dados - Listas

Estrutura de uma Lista Simplesmente Encadeada

Cada nó tem:

1. Um campo para armazenar o **valor** (dado)
2. Um campo para armazenar a **referência para o próximo nó**



Estrutura de dados - Lista Simplesmente Encadeada

◆ Inserir no final

yaml

ANTES:



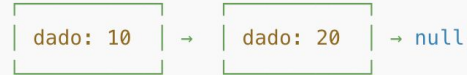
DEPOIS de inserir 30:



◆ Inserir no início

yaml

ANTES:



DEPOIS de inserir 5:



Estrutura de dados - Lista Simplesmente Encadeada

Onde utilizar?

- **Implementação de Filas (Queue)** – onde só precisamos enfileirar no fim e desenfileirar no início.
- **Implementação de Pilhas (Stack)** – usando a cabeça da lista para push e pop.
- **Tabelas de Hash** – para resolver colisões com listas encadeadas.
- **Listas de Adjacência em Grafos** – armazenar vizinhos de cada vértice.
- **Estruturas auxiliares em algoritmos** – como percorrer árvores ou listas de tarefas.

Estrutura de dados - Lista Simplesmente Encadeada



Onde utilizar?

1. Sistemas Operacionais / Gerenciamento de Memória

- Usadas para gerenciar **blocos de memória livres** em alocação dinâmica — o sistema mantém uma lista encadeada para rastrear espaços disponíveis na memória.

Blockchain (Semelhante a uma LSE)

- Cada bloco aponta para o anterior, formando uma cadeia linear — essencialmente uma “lista encadeada” com garantia criptográfica.

Estrutura de dados - Lista Simplesmente Encadeada

Quando e onde não utilizar?

- **Acesso por índice** (planilhas, arrays indexados)
- **Big data/cálculos** (processamento matemático intensivo)
- **Sistemas embarcados** (memória limitada)
- **Busca/ordenação** (operações que precisam de acesso aleatório)
- **Tempo real** (performance previsível necessária)

Problema: Acesso $O(n)$, overhead de memória, cache ruim

Comparando com outros tempos

- $O(1)$ → constante (independe do tamanho da entrada). Ex: acessar `lista[0]`.
- $O(\log n)$ → cresce devagar. Ex: busca binária em vetor ordenado.
- $O(n)$ → cresce linearmente. Ex: percorrer todos os elementos de uma lista.
- $O(n^2)$ → cresce rápido (quadrático). Ex: percorrer pares de elementos (loops aninhados).

Regra Prática

Use quando **inserções/deleções** são frequentes e tamanho varia. Evite quando precisa de **acesso direto** ou **performance previsível**.

Estrutura de dados - Listas

- **Exercícios**

Estrutura de dados - Lista Simplesmente Encadeada



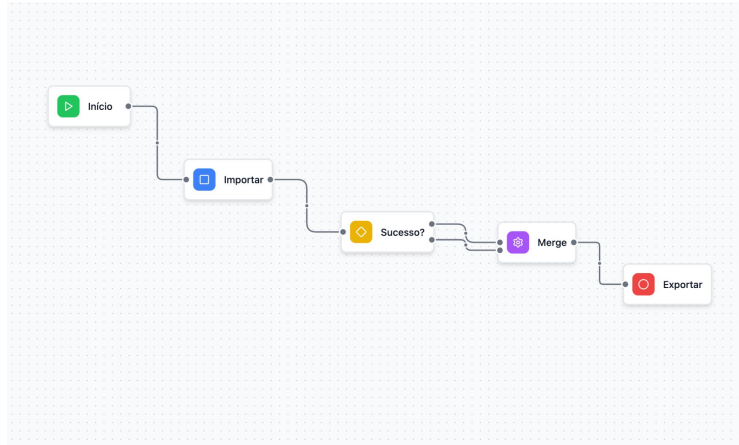
Estrutura de uma Lista Simplesmente Encadeada

Código de exemplo: <https://trinket.io/java/5fc734e7edd1>

Estrutura de dados - Listas

- **Caso de uso**

Estrutura de dados - Lista Simplesmente Encadeada



<https://preview-reactflow-editor-kzmljghseurgg3cfw9xv.vusercontent.net/>

<https://drive.google.com/file/d/1gGuoehYPC9u-MfvyRvQUWy6y093KIC06/view?usp=sharing>

<https://jsoneditoronline.org/>

<https://bucket-squid-98193005.figma.site/>

Estrutura de dados - Lista Simplesmente Encadeada

Estruturas Principais:

1. **FlowNode**: Representa um nó do fluxo com ID, tipo, posição e dados
2. **FlowEdge**: Representa uma conexão entre dois nós
3. **NodeList**: Lista encadeada customizada para armazenar nós
4. **EdgeList**: Lista encadeada customizada para armazenar arestas
5. **Flow**: Classe principal que gerencia todo o fluxo

Funcionalidades Implementadas:

- **Adição/Remoção** de nós e arestas
- **Busca** por ID, origem ou destino
- **Navegação** no grafo (predecessores/sucessores)
- **Traversal** das listas encadeadas
- **Exibição** formatada do fluxo

Estrutura de dados - Lista Simplesmente Encadeada

Vantagens desta Implementação:

1. **Memória Eficiente:** Aloca apenas o necessário
2. **Inserção Rápida:** $O(1)$ no final da lista
3. **Flexibilidade:** Fácil de estender com novos tipos
4. **Navegação de Grafo:** Encontrar conexões rapidamente

Exemplo de Uso:

O método `main()` demonstra como criar um fluxo completo com:

- 6 nós (início, processamento, decisão, ações, fim)
- 6 conexões entre os nós
- Navegação pelos sucessores e predecessores

Estrutura de dados - Listas

- **Exercícios**

Estrutura de dados - Lista Simplesmente Encadeada



Praticar!

<https://trinket.io/java/df8ca16ce864>

Estrutura de dados - Listas

- **Listas duplamente encadeadas**

Estrutura de dados - Listas

Estrutura de uma Lista Duplamente Encadeada

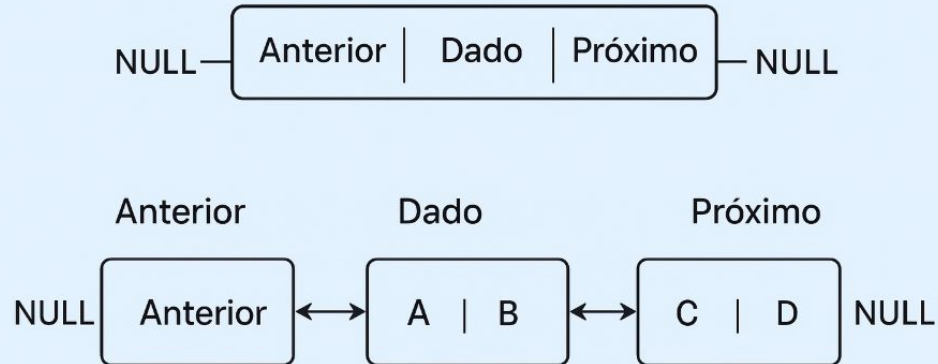
Antes de entrar em listas duplamente encadeadas:

Comparação Simples

- **Lista Simplesmente Encadeada (LSE):**
 - Só anda em **uma direção**
 - Cada nó guarda **1 ponteiro** (para o próximo)
 - Mais **simples** e ocupa **menos memória**
- **Lista Duplamente Encadeada (LDE):**
 - Anda em **duas direções**
 - Cada nó guarda **2 ponteiros** (próximo e anterior)
 - Mais **flexível**, mas ocupa **mais memória** e é **mais complexa**

Estrutura de dados - Listas

Estrutura de uma Lista Duplamente Encadeada

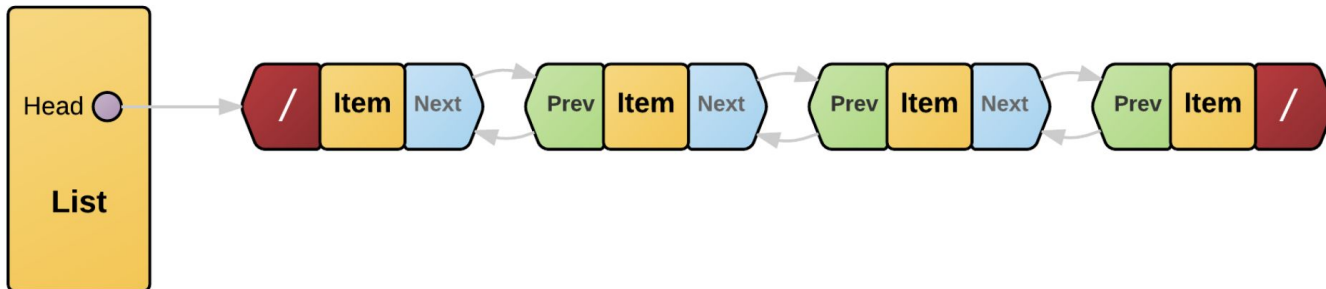


Estrutura de dados - Listas

Estrutura de uma Lista Duplamente Encadeada

1. Estrutura de nós conectados entre si.
2. Cada nó contém:
 - **Dado** (informação armazenada)
 - **Ponteiro para o próximo nó**
 - **Ponteiro para o nó anterior**

Isso permite **navegar tanto para frente quanto para trás**.



Estrutura de dados - Lista Duplamente Encadeada



Onde utilizar?

Navegação em Navegadores e Exploradores de Arquivos

- Navegadores utilizam LDE para permitir os botões "voltar" e "avançar", navegando suavemente entre páginas visitadas

Funções de Undo/Redo (Desfazer/Refazer)

- Editores de texto, IDEs e softwares como Photoshop ou Word implementam histórico de ações com LDE, permitindo avançar e retroceder em alterações

Playlists em Aplicativos de Mídia

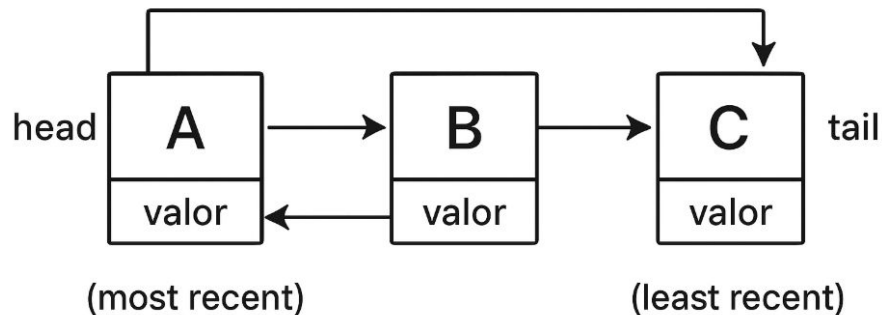
- Em players de música (Spotify, Apple Music, etc.), LDE facilitam navegar para a próxima ou a música anterior com facilidade

Exemplo: LRU Cache com Lista Duplamente Encadeada

Estrutura

- Nó da LDE guarda **chave**, **valor**.
- A cabeça (head) da lista = item mais **recente**.
- A cauda (tail) da lista = item mais **antigo** (pronto para ser removido).

LRU Cache



Estrutura de dados - Lista Duplamente Encadeada



Quando e onde não utilizar?

Quando memória é limitada

- Cada nó carrega **dois ponteiros** extras, o que aumenta o consumo de memória.
- Em sistemas embarcados ou dispositivos com pouca RAM, isso pode ser inviável.

Quando o acesso rápido por índice é necessário

- LDE não permite acesso direto como vetores/arrays ($O(1)$), apenas percorrendo nó a nó ($O(n)$).
- Se a aplicação precisa consultar muito por posição (ex: `lista[1000]`), é ruim.

Quando só percorremos em uma direção

- Se a navegação reversa não é necessária, uma **lista simplesmente encadeada** já resolve, ocupando menos memória e sendo mais simples de implementar.

Quando a estrutura tem tamanho fixo e previsível

- Se sabemos o tamanho máximo (ex: lista de 1000 elementos sempre), um **array** é melhor: ocupa menos memória e é mais eficiente em cache.

Estrutura de dados - Lista Duplamente Encadeada



Quando e onde não utilizar?

Em aplicações com foco em cache e performance de CPU

- Arrays aproveitam melhor a **localidade de memória** (cache da CPU).
- LDE pode ser bem mais lenta em cenários de alto desempenho, mesmo tendo operações $O(1)$ de inserção/remoção.

Quando a complexidade do código importa

- Implementar, depurar e manter LDE é mais difícil.
- Se a simplicidade for prioridade, muitas vezes um array ou LSE basta.

Estrutura de dados - Listas

- **Exercícios**

Estrutura de dados - Lista Duplamente Encadeada



Estrutura de uma Lista Duplamente Encadeada

Código de exemplo: <https://trinket.io/java/cc843db34f78>

Simulado

